

Agent Airlock

Give AI agents room to move, not room to break the rules.

Room to move. Rules that hold.

Agent Airlock checks each mission before takeoff, pauses risky actions for approval, and blocks forbidden actions before they reach real systems.

AGENT PROPOSES



POLICY DECIDES



EXECUTE OR STOP

THE PROMPT DESCRIBES THE DESTINATION. AGENT AIRLOCK CLEARS THE SAFE PATH.

AI agents have hands now

They do not only recommend actions; they can reach systems that matter.

CODE

Change repositories

Edit files, create branches and prepare pull requests.

DATA

Move information

Read logs, drafts and enterprise context—then display or publish it.

SUPPLY CHAIN

Choose dependencies

Modify manifests, restore packages and introduce new risk.

OPERATIONS

Trigger effects

Push, publish, invoke models and call external tools.

The risk is not that the agent can reason. The risk is that reasoning can become an effect.

Instructions shape behavior. They do not create a security boundary.

Repository and personal instructions are useful guidance, but the agent still needs a separate permission layer.

CONTROL	GOOD FOR	WHY IT IS NOT ENFORCEMENT
Prompt	Describing the desired outcome	Untrusted text can request exceptions or conflict with policy
Repo instructions	Coding conventions and workflow	Owned with the workload and interpreted by the model
Personal instructions	Developer preferences	Different users can carry different guidance
Airlock policy	Permission for protected effects	Trusted code evaluates a structured action before execution

Guidance can influence the proposal. Only the guarded path controls the effect.

Put a controlled boundary between intent and effect

Let the agent explore inside; guard the operations that leave the chamber.

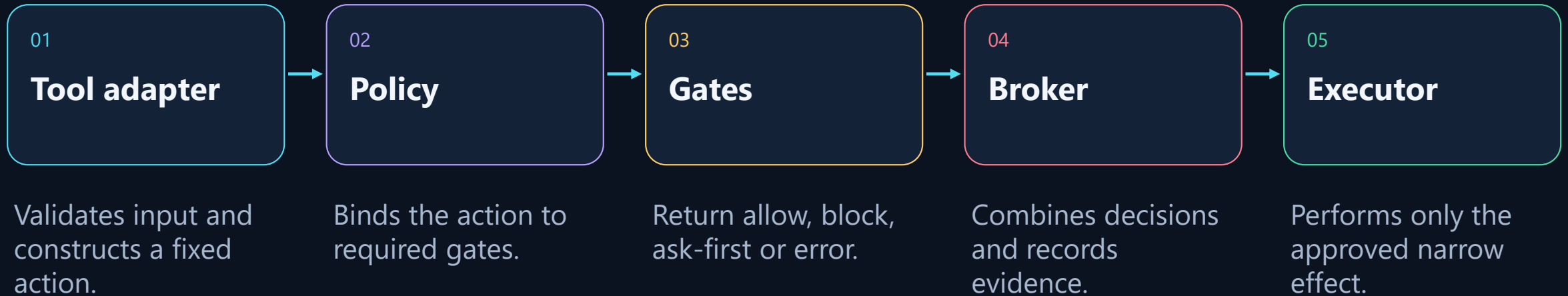


Nothing leaves until it clears policy.

The airlock is a transition point: propose freely, execute only after policy clears the action.

The system is structured around one enforcement owner

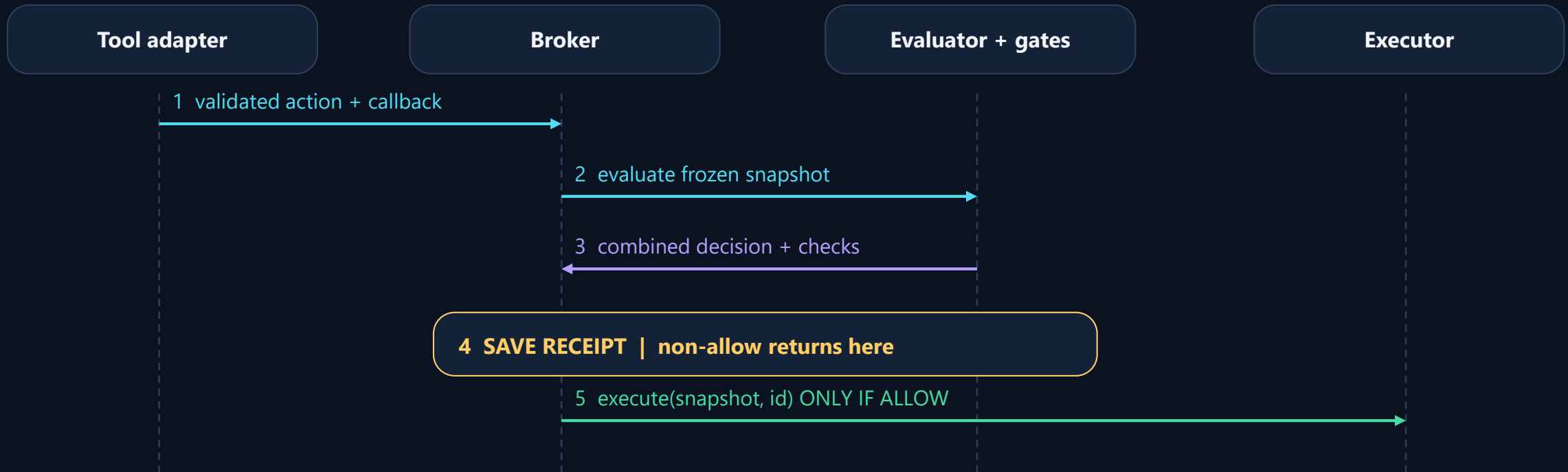
Each component has one job; only the broker controls execution.



The model can request a supported action. It cannot supply or invoke the executor.

What happens when an action reaches Airlock

The exact bytes that are checked are the bytes the executor receives.



Decision receipt first. Executor second. Any non-allow result returns without opening the door.

Every gate uses the same decision contract

Different risks compose because their outputs are deliberately boring.

FORBIDDEN

BLOCK

Policy rejects the action.
Execution never starts.

CHECK FAILED

ERROR

A required control failed.
Fail closed before the
effect.

HUMAN NEEDED

ASK-FIRST

Pause for review. Today
the prototype stops
here.

ALL CLEAR

ALLOW

Every required gate
passed. The broker may
execute.

A green result from one gate cannot cancel a red result from another.

Sensitive Information Protection

Prevent secrets, tokens, personal data and internal-only content from leaking through drafts, logs or display text.

PUBLISH_DRAFT / THREE POLICY-BOUND CHECKS

Inspect outbound text before any publish effect.

CHECKS

Secrets and contextual tokens → block
PII patterns → ask-first
INTERNAL-ONLY label → ask-first

POLICY RESULT

Only clean or safely remediated content publishes

► SCREEN RECORDING

```
npm run demo -- --gate sensitive-information
```

CLEAN

published

SECRET

blocked

PII / LABEL

review

Recording slot: recordings/sensitive-information.mp4

Prompt overrides do not beat the detector; blocked and review-required cases create no publish artifact.

Dependency Risk

Review the proposed package and version before editing a manifest or restoring dependencies.

REVIEW_DEPENDENCY_CHANGE

**Move dependency review
ahead of the irreversible work.**

CHECKS

Package identity and proposed version
Reviewed synthetic blocklist
Advisory evidence path when available

POLICY RESULT

Blocked proposal → no edit, no restore

► SCREEN RECORDING

```
npm run demo -- --gate dependency-risk
```

PACKAGE

MSAL

VERSION

demo-blocked

EFFECT

not started

Recording slot: recordings/dependency-risk.mp4

The demo is a deterministic synthetic policy case—not a CVE claim about the sample repository.

Model Catalog

Constrain model selection by task and data classification before inference is requested.

SELECT_MODEL

Choose only reviewed model routes for the declared context.

CHECKS

Task type and caller-supplied data class
Default, permitted override or blocked entry
Catalog status and policy

POLICY RESULT

Default selects; override pauses; blocked model stops

► SCREEN RECORDING

```
npm run demo -- --gate model-catalog
```

DEFAULT

selected

OVERRIDE

review

PUBLIC

blocked

Recording slot: recordings/model-catalog.mp4

The prototype records a local selection; it does not switch Copilot's active model or invoke inference.

Online-Write Intent

Detect explicit requests to push or publish before an online-write tool is selected.

CHECK_INTENT

Treat forceful wording as intent—not as authorization.

CHECKS

Supplied prompt text
Reviewed online-write patterns
Local-review versus external-write intent

POLICY RESULT

"/yolo git push ..." is still blocked

► SCREEN RECORDING

```
npm run demo -- --gate intent
```

LOCAL REVIEW

cleared

PUSH REQUEST

blocked

SHELL

not executed

Recording slot: recordings/intent.mp4

This gate checks text routed through Airlock; it is not an operating-system shell sandbox.

Trending Cost

Surface local usage trends with policy context before cost becomes invisible.

TRENDING_COST

Make operational usage visible through the same policy surface.

CHECKS

Local CLI usage-style SQLite data
Month-to-date cost and current-day usage
Configured advisory threshold

POLICY RESULT

Above threshold reports an advisory—not a hard block

► SCREEN RECORDING

```
npm run demo -- --gate trending-cost
```

BELOW

reported

ABOVE

advisory

MISSING

explicit

Recording slot: recordings/trending-cost.mp4

Synthetic disposable data only; this is not billing, spend reservation or account-wide enforcement.

Every decision leaves evidence

Receipts explain what was checked and whether the guarded executor ran.

RECEIPT FIELD	QUESTION ANSWERED	IMPORTANT LIMIT
Policy ID + hash	Which reviewed policy applied?	Not whole-plugin integrity
Gate checks	Which controls ran and decided?	Detection remains bounded
Safe findings	What rule and location matched?	Raw sensitive value is omitted
Execution state	Did this executor start?	Does not prove no alternate route existed
Artifact / error	What happened after allow?	External effects may require stronger attestation

Evidence is part of the control path, not an after-the-fact log message.

Adding a gate is intentionally small

New checks plug into a stable result contract without owning the side effect.

Gate shape / safe result

```
{
  id: "reviewed-check",
  failureReason: "check-failed",
  async evaluate(action) {
    return {
      decision: "allow",
      reason: "safe-reason-code",
      findings: []
    };
  }
}
```

Read-only evaluator

The gate inspects the action; it never performs the protected operation.

Explicit composition

Register trusted code, bind it in policy and test all decision paths.

The extension point is the gate. The enforcement owner remains the broker.

What the prototype enforces—and what it does not yet

Airlock controls supported actions routed through Airlock. Alternate permitted routes remain the system-level gap.

AIRLOCK ROUTE



OTHER ROUTES



Another MCP or shell route cannot override an Airlock block; it can bypass Airlock if the host still permits that route.

The path from prototype to shared enforcement

The next milestone is not more warnings. It is making protected routes unavoidable.

PRIORITY	CAPABILITY	PROOF
1	Mandatory host or destination mediation	Alternate-route coverage tests
2	Approval redemption bound to the exact action	Expiry, replay and mutation rejection
3	Protected policy distribution and integrity	Missing or invalid controls fail closed
4	Narrow real-world executors	Destination, credential and retry guarantees
5	Broader gate catalog	New risks compose without weakening existing checks

Control the door first. Then add more detectors behind it.

What we want the team to remember

Autonomy and control are not opposites when permission lives at the action boundary.

Give AI agents room to move, not room to break the rules.

The prompt describes the destination.
Agent Airlock clears the safe path.

PROPOSE FREELY. CHECK CONSISTENTLY. EXECUTE DELIBERATELY.